XRPL Validator — Daily Engineering Report · 2026-08-14

Period: 2026-08-14 00:00 to 24:00 US Eastern (2026-08-14 04:00 to 2026-08-15 04:00 UTC). Generated 2026-10-21 00:47 EDT by report generator v3.0. Sources: the XRP Ledger (ledger headers and amendments, read from a reference rippled node), the validator's signing log and receipt logs, the deployment pipeline's gate, soak and health records, the git history of xrpl-validator and the XRPL Rust SDK, and data.xrpl.org. Scope: XRPL work only.

Revised on 2026-10-21 in report format v3; the original is kept in this repository's history. Built only from records that exist for this day. Available: xrpl-validator merge records (they begin 2026-07-04). Not available: ledger headers (the reference node holds ledgers from 2026-10-04 22:52 UTC); signing records (they begin 2026-09-19); Step 1 receipt records (they begin 2026-08-29); Steps 2–5 receipt records (they begin 2026-10-09); deployment records (they begin 2026-09-11); amendment data; data.xrpl.org agreement scores (scored days begin 2026-04-04; some days have none).

Status

Not assessed — no signing records for this day.

Criteria. Normal: coverage outside planned maintenance at or above the 99.9% objective, every validation agreed with the network's validated ledger, and no engine receipt is open. Degraded: coverage below the objective, or an open engine receipt. Impaired: any validation that disagreed with the network, or coverage below 90%.

Executive summary

- **Validator:** no signing records for this day (they begin 2026-09-19).
- **Native engine:** no receipt records for this day (they begin 2026-08-29).
- **Changes:** 0 deployments, 0 commits merged into main.

Service levels

No signing records for this day (they begin 2026-09-19).

Native engine conformance

No receipt records for this day (Step 1 records begin 2026-08-29, Steps 2–5 records 2026-10-09).

Live counters are kept only for the current run; none exist for a past day.

Receipt register

No receipt records for this day (Step 1 records begin 2026-08-29).

Incidents

Not assessed: no signing records for this day.

Change log

Deployments

No deployment records before 2026-09-11.

Merged into main

No merges into main.

Findings fixed

No engine findings merged into main this day.

Other engineering changes

- **fix(xrpl-ledger): measure the REVERSE pass's consumption precisely too** (`45a7092`, xrpl-validator) `44c20d9` fixed the forward carry and I did not notice the same defect one level up. `measure_hop` worked out how much a trial consumed by differencing a GRANTED balance — 16 significant digits TOTAL, so the grant itself eats the low end of the answer — while `cross_engine_to` was already RETURNING the precise remaining input and the code threw it away with `let (rw, _, _) = ...`.
- **fix(xrpl-ledger): the precise measurement must account for the fee the walk debits** (`1f31546`, xrpl-validator) A defect in `45a7092`. Its precise path is `granted - rem_in`, but `rem_in` tracks only the NET the walk moved — the input transfer fee is debited SEPARATELY by `line_adjust`. So on a fee-bearing hop `precise` is the net while the differenced balance `d` is the gross, the two disagree by the WHOLE FEE (1e-3 for a 1.001 issuer, a thousand times the 1e-9 agreement threshold), and the guard rejected the precise value …
- **fix(xrpl-ledger): ask the walk what a strand spent, don't difference the balance** (`8d8d2e6`, xrpl-validator) Third instance of the defect `44c20d9` fixed for a hop's carry and `45a7092` for the reverse pass — and exactly the `signed_add(before, !after)` shape 45a7092's note said to go looking for. `strand_pass` reported its input consumption by differencing the sender's spend-leg balance, and a balance holds 16 significant digits TOTAL, so whatever the sender already holds of the spend currency eats the low end of the …
- **feat(xrpl-ledger): one AMM consumption per payment-engine iteration** (`4566c4e`, xrpl-validator) "At any payment engine iteration, AMM offer can only be consumed once" (BookStep.cpp:818). When a payment wants more pool liquidity than one AMM offer gives, rippled ENDS the pass and `flow()` re-enters the strand for the next one — and every iteration writes the maker's residual, so N iterations are N roundings. We took Fibonacci slices inside a single pass and rounded once.
- **fix(xrpl-ledger): the bridge's direct head prices a partial fill at the filed rate** (`7f8286b`, xrpl-validator) The rule `e2a9c99` gave the two bridge legs, never given to the third fill site in the same function. An offer carries two rates: the one it is FILED under (low 64 bits of `BookDirectory`, a 16-digit mantissa fixed at creation) and its

current TakerPays/TakerGets, which drifts every time a residual is re-rounded. rippled prices a PARTIAL fill at the filed rate and a FULL one at the amounts.

- **fix(xrpl-ledger): limitOut takes the STRICT canonicaliser when the drops decide** (`ff1e752`, xrpl-validator) rippled's own comment on `TOffer::limitOut` (Offer.h:200): "It turns out that the ceil_out implementation has some slop in it, which ceil_out_strict removes." So an out-limited fill goes `ceilOutStrict` -> `mulRoundStrict` -> `canonicalizeRoundStrict`, not `mulRound`. After dividing down to offset -1 the two differ by one term (STAmount.cpp:1477 vs :1516): canonicalizeRound value += 9; canonicalizeRoundStrict value …
- **fix(xrpl-ledger): rippleSend moves the sender's line ONCE, with the gross** (`bc756dd`, xrpl-validator) rippled's `rippleSend` (View.cpp) is one credit per party: saActual = saAmount + saTransitFee; rippleCredit(issuer, receiver, saAmount); // receiver gets the NET rippleCredit(sender, issuer, saActual); // sender pays the GROSS
- **fix(xrpl-ledger): every payment strand runs ONE PASS — corpus byte parity 0 hits** (`950e63e`, xrpl-validator) The third and last part of the iteration model, and the SAME `if multi` gate the rounds carried in `4566c4e`. A pass ends when an offer is CONSUMED and `flow()` re-enters the strand; `single_pass` already implements that, but it was switched on only for multi-strand payments, so a lone strand walked the whole book in one call and MERGED what rippled does in separate iterations.
- **fix(xrpl-ledger): NFTokenAcceptOffer weighs the buyer's funds — and the probe lets it see them** (`9c0d845`, xrpl-validator) 9 of the 13 divergences in the 2026-08-14 fresh sweep were one cluster: `our_ter=tecOBJECT_NOT_FOUND` against `net_ter=tecINSUFFICIENT_FUNDS`, 1 mutation each side. TWO defects stacked, and fixing either alone measures WORSE than fixing neither.

Independence program

No independence record for UTC day 2026-08-14; records begin 2026-10-20.

Network

No ledger records for this day: the reference node holds ledgers from 2026-10-04 22:52 UTC (#107,436,647).

Amendment watch

No amendment records for this day.

Open items

None.

Corrections to earlier reports

None.

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **PayChannel Merkle Batch Claims** (gaming) Add an lsfMultiClaim flag and sfClaimantRoot (Hash256 Merkle root over {Destination, CumulativeAmount} leaves) to PayChannel, plus a new PayChannelBat
- **Commit-Reveal Offers Against Front-Running** (dex) Add two transaction types: OfferCommit, creating an owner-reserved ltOFFER_COMMIT object holding sfCommitHash (Hash256 = sha512Half(TakerPays||TakerGe
- **Credentialed Sponsor-Onboarding for MPTs** (tokens) Extend MPTokenAuthorize with optional sfSponsor (AccountID) and sfSponsorCredentialType SFields: when present, preclaim looks up an existing Credentia

Appendix — commits

Committed	Merged into main	Repo	Commit	Subject
08-14 04:06	—	xrpl-validator	`45a7092`	fix(xrpl-ledger): measure the REVERSE pass's consumption precisely too
08-14 14:26	—	xrpl-validator	`1f31546`	fix(xrpl-ledger): the precise measurement must account for the fee the walk debits
08-14 16:31	—	xrpl-validator	`8d8d2e6`	fix(xrpl-ledger): ask the walk what a strand spent, don't difference the balance
08-14 17:30	—	xrpl-validator	`4566c4e`	feat(xrpl-ledger): one AMM consumption per payment-engine iteration
08-14 19:34	—	xrpl-validator	`7f8286b`	fix(xrpl-ledger): the bridge's direct head prices a partial fill at the filed rate
08-14 20:37	—	xrpl-validator	`ff1e752`	fix(xrpl-ledger): limitOut takes the STRICT canonicaliser when the drops decide
08-14 21:31	—	xrpl-validator	`bc756dd`	fix(xrpl-ledger): rippleSend moves the sender's line ONCE, with the gross
08-14 22:34	—	xrpl-validator	`950e63e`	fix(xrpl-ledger): every payment strand runs ONE PASS — corpus byte parity 0 hits
08-15 01:40	—	xrpl-validator	`9c0d845`	fix(xrpl-ledger): NFTokenAcceptOffer weighs the buyer's funds — and the probe lets it see them

Times in UTC. A commit appears on the day it was committed to main or merged into it.